

Pietro

Possiamo eliminare i ghost mantenendoli però come interfaccia, cioè come una simil-instance della classe firm da mettere negli elenchi come variabile complessa con più campi, alcuni dei quali sono riferimenti alla memoria condivisa?

ChatGPT

Sì. Anzi, **credo sia una direzione architetturale molto promettente** per il modello attuale.

Non eliminerei il concetto di ghost: eliminerei i **ghost Repast4Py come oggetti Firm completi**. Al loro posto introdurrei un oggetto Python molto leggero, per esempio `FirmProxy`, che presenta al resto del codice quasi la stessa interfaccia di una `Firm`.

L'idea sarebbe:

```
class FirmProxy:
    __slots__ = (
        "uid",
        "address",
        "sector",
        "country",
        "productionType",
    )

    def __init__(
        self,
        uid,
        address,
        sector,
        country,
        productionType,
    ):
        self.uid = uid
        self.address = address
        self.sector = sector
        self.country = country
        self.productionType = productionType

    @property
    def outputWarehouse(self):
        return Firm.sharedOutputWarehouse[self.address]

    @property
    def targetAddedValue(self):
        return Firm.sharedTargetAddedValue[self.address]
```

Quindi negli elenchi dei supplier potremmo avere indifferentemente:

```
Firm locale
FirmProxy remota
FirmProxy remota
Firm locale
...
```

e il codice continuerebbe a fare, per esempio:

```
supplier.country
supplier.sector
supplier.address
supplier.outputWarehouse
supplier.targetAddedValue
```

senza preoccuparsi se `supplier` è una vera `Firm` o una proxy.

Perché nel nostro caso è particolarmente adatto

Il passaggio alla shared memory ha già fatto gran parte del lavoro concettuale necessario. Per un supplier remoto, i dati che cambiano continuamente e che ci interessano sono già fuori dal ghost:

```
outputWarehouse —> sharedOutputWarehouse[address]
targetAddedValue —> sharedTargetAddedValue[address]
```

Il ghost Repast4Py è diventato quindi, in larga misura, **un involucro Python che ci permette di identificare la Firm e raggiungere quelle celle.**

Inoltre abbiamo già deciso che i ghost sono passivi: non eseguono `supply()`, `demand()` ecc. Questo rende molto meno necessario che siano vere istanze complete di `Firm`.

Possiamo andare anche oltre

Per i supplier remoti, probabilmente non abbiamo nemmeno bisogno di `uid` completo. Potrebbe bastare una struttura del tipo:

```
class SupplierProxy:
    __slots__ = (
        "address",
        "sector",
        "country",
    )

    def __init__(self, address, sector, country):
        self.address = address
        self.sector = sector
        self.country = country

    @property
    def outputWarehouse(self):
        return Firm.sharedOutputWarehouse[self.address]

    @property
    def targetAddedValue(self):
        return Firm.sharedTargetAddedValue[self.address]
```

Questo oggetto sarebbe minuscolo rispetto a una `Firm`.

E qui c'è un ulteriore vantaggio: `sector`, `country` e forse `productionType` sono **immutabili**. Non devono necessariamente essere attributi Python duplicati in centinaia di migliaia di proxy. Potremmo avere altri array condivisi:

```
firmSector[address]
firmCountry[address]
firmProductionType[address]
targetAddedValue[address]
outputWarehouse[address]
```

e allora una proxy potrebbe contenere praticamente **solo** `address`:

```
class SupplierProxy:
    __slots__ = ("address",)

    def __init__(self, address):
        self.address = address

    @property
    def country(self):
        return Firm.sharedCountry[self.address]

    @property
    def sector(self):
        return Firm.sharedSector[self.address]

    @property
```

```
def outputWarehouse(self):
    return Firm.sharedOutputWarehouse[self.address]

@property
def targetAddedValue(self):
    return Firm.sharedTargetAddedValue[self.address]
```

Questa è precisamente la tua idea di «**simil-instance**» della classe **Firm**: dal punto di vista del codice utilizzatore si comporta come un supplier, ma in realtà è soltanto una vista molto leggera sui dati condivisi.

La conseguenza più interessante

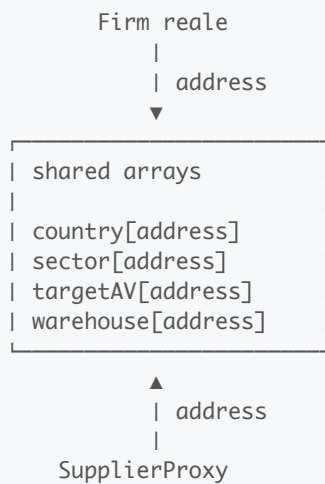
A quel punto potremmo probabilmente eliminare anche:

```
context.request_agents(...)
```

per questi supplier e quindi tutta la costruzione dei ghost Repast4Py.

Ogni rank dovrebbe conoscere soltanto gli `address` dei supplier che possono comparire nei suoi elenchi. Le proprietà necessarie verrebbero recuperate direttamente dagli array condivisi.

Avremmo:



Non sarebbe più propriamente un *ghost agent*. Sarebbe una **proxy/view di un agent remoto**.

E questo potrebbe ridurre la memoria in maniera molto forte, perché oggi ogni ghost viene costruito attraverso il normale costruttore `Firm`, che contiene anche strutture che un supplier passivo non utilizzerà mai — comprese, nel nostro codice attuale, varie liste di 64 elementi.

La parte delicata non è quindi la shared memory: **quella l'abbiamo già risolta**. La parte da verificare nel notebook è esattamente **quali attributi di un supplier vengono effettivamente letti dopo `createFirmSubsets()`**. Da quella lista possiamo progettare una `SupplierProxy` minimale e, idealmente, fare in modo che il resto di `selectSuppliers()` non debba quasi accorgersi della sostituzione.

Questa mi sembra anche una modifica da fare **prima** di pensare ad altre ottimizzazioni prestazionali: non cambia l'algoritmo economico, ma elimina una duplicazione strutturale potenzialmente enorme.